

Technical and Editorial Review

Object-Oriented Design Patterns Seminar Report

August 28, 2026

Overall assessment

The report is clearly organized and the Facade and Strategy chapters give useful problem–solution narratives, trade-off discussions, and compilable C++11 demonstrations. The manuscript itself compiles successfully to a 30-page PDF, and the checked Facade programs produce the same device-output sequence.

The report is not yet submission-ready. The largest scientific and technical problem is that the navigation examples are described as route-calculation algorithms even though they ignore both inputs and print fixed text. The diagrams also contradict the code at precisely the ownership and constness points that the report aims to teach. In addition, the Decorator chapter is materially less complete than the other two chapters and does not satisfy several explicit per-pattern requirements. Placeholder contribution text and citation gaps remain.

No formal mathematical derivations, coordinate transformations, inverse-trigonometric expressions, or sign-sensitive formulas occur in the manuscript. Branch and quadrant checks were therefore not applicable. The audit did check all quantitative examples, type signatures, ownership statements, lifetime claims, diagrams, implementation listings, appendices, and references. The required notation and terminology drift check found several concrete conflicts, detailed below.

Technical and evidence findings

1. The Strategy implementations do not calculate routes

Classification.

Unsupported claim; major.

Location.

`main.tex`, “Real-world problem: navigation route planning” and “Strategy interface and concrete algorithms” (especially lines 592–612 and 748–849); the four files in `Seminar/Strategy/src/strategy/`.

Problem.

Each concrete `buildRoute(origin, destination)` ignores `origin` and `destination`, returns no route, and prints a fixed mode description, distance, and travel time. The same “25 mins (18.5 km)” is printed by `DrivingStrategy` for every pair of locations. Compiling the

supplied Strategy program with `-Wall -Wextra -Wpedantic` produces eight unused-parameter warnings. Calling these classes route-calculation algorithms overstates what the evidence demonstrates.

Why it matters.

The central example supports runtime dispatch, but it does not support claims about route calculation, algorithm-specific data, traffic, timetables, elevation optimization, estimated performance, or the testing of real routing behavior. Readers may confuse a Strategy-pattern skeleton with an implemented algorithm.

Suggested fix.

Either label the classes explicitly as pedagogical stubs that demonstrate interchangeable behavior only, and remove the fabricated quantitative outputs, or implement a minimal input-dependent contract. For example, return a `RouteResult` containing a path, distance, duration, and status, and make each strategy consume its inputs or injected data source. The testing and performance discussion should then be limited to what the implementation actually demonstrates.

2. Diagram types and relationship semantics contradict the code

Classification.

Definite internal inconsistency; major.

Location.

Decorator diagram in `main.tex`, lines 202–220; Figure `fig:facade-theater`; Figure `fig:strategy-navigation`; Strategy discussion at lines 743–745 and 970–974.

Problem.

The Decorator diagram gives `wrapped` the type `Beverage*`. The listing instead uses `std::unique_ptr<Beverage>` and claims ownership. The home-theater diagram shows value-like fields such as `Amplifier amp`, whereas the implementation stores `Amplifier&` and four other references. The navigation diagram shows `RouteStrategy*` and a setter accepting that type, whereas the code uses `constRouteStrategy*`. Finally, the hollow diamond is described first as delegation through a non-owning pointer and later as “composition from Context to Strategy.” Those terms are not interchangeable in UML ownership semantics.

Why it matters.

Ownership, lifetime, mutability, and substitutability are central teaching points in this report. Removing `unique_ptr`, reference, and `const` qualifiers from the diagrams teaches a different design from the code. Calling a non-owning association “composition” can also imply that `Navigator` controls the strategy lifetime, which it does not.

Suggested fix.

Make every implementation diagram match the exact code types, or state explicitly that types are simplified and add ownership notes. Replace the Facade relation label “wraps” with “uses” or “coordinates.” For Strategy, use a directed non-owning association rather than a composition claim, preserve `const RouteStrategy*`, and revise line 974 to “a non-owning association from Context to Strategy.” Reserve “object composition” for the broad design principle and distinguish it from UML composition.

3. The const-correctness explanation is technically inaccurate

Classification.

Definite error.

Location.

`main.tex`, “Const correctness and demonstration parameters,” lines 860–866.

Problem.

The report says that both the pointer and `buildRoute` are `const`. The declaration `const RouteStrategy* strategy` instead denotes a modifiable pointer to a const-qualified `RouteStrategy`, not a const pointer. Indeed, `setStrategy` changes the pointer. Moreover, a const-qualified member function does not absolutely guarantee that the object performs no mutation, because mutable state and external effects remain possible.

Why it matters.

Pointer constness is a common source of C++ errors, and this paragraph is presented as instruction on that exact topic.

Suggested fix.

Replace the sentence with: “`strategy` is a modifiable pointer through which the selected strategy is treated as const; `Navigator::buildRoute` is const-qualified and therefore does not modify the Navigator’s ordinary data members. The interface permits only the strategy’s const-qualified `buildRoute` operation.”

4. The Decorator listing has an unstated C++14 requirement

Classification.

Definite compatibility issue if C++11 is intended; otherwise implementation ambiguity.

Location.

`main.tex`, “C++ implementation sketch,” lines 228–262.

Problem.

The listing uses `std::make_unique`, which is available from C++14, while the other two implementations are explicitly presented and tested as C++11. Compiling the shown Decorator code under `g++ -std=c++11` fails at both `std::make_unique` calls. The listing also omits the required `<memory>`, `<string>`, and `<utility>` headers if it is meant to be independently compilable.

Why it matters.

The report otherwise presents a coherent C++11 seminar codebase. An unstated language-version change prevents readers from reproducing one of the three examples.

Suggested fix.

State that the Decorator example requires C++14 and show its includes, or provide a C++11-compatible construction. Prefer adding a small complete source file and a documented compile command so the example is tested in the same way as Facade and Strategy.

5. Reference members exclude an ordinary null state but do not guarantee validity

Classification.

Likely misleading claim.

Location.

`main.tex`, Facade implementation discussion, lines 446–449.

Problem.

The report says references “guarantee non-null dependencies.” Reference members make an explicit null value unavailable in normal use, but they do not prove that the referred objects remain alive, and invalid references can still be formed through undefined behavior. The next sentence correctly acknowledges the lifetime requirement.

Why it matters.

The current phrasing can be read as a runtime safety guarantee even though the implementation has no lifetime enforcement.

Suggested fix.

Say that references “make the absence of a dependency unrepresentable in ordinary well-defined construction, but remain non-owning; the subsystem objects must outlive the Facade.” Consider deleting `rvalue` overloads or documenting construction constraints if this example is expanded.

6. Header self-containment is incomplete

Classification.

Likely portability issue.

Location.

`Seminar/Facade/src/facade/Projector.h`, line 10, and the corresponding naive header; `HomeTheaterFacade.h`, line 22.

Problem.

`Projector.h` uses `std::string` but does not include `<string>`. `HomeTheaterFacade.h` also uses `std::string` while obtaining its declaration only through other project headers. The current include order happens to compile, but it relies on transitive includes.

Why it matters.

A header should compile when included on its own; otherwise unrelated include-order changes can break the code.

Suggested fix.

Add `#include <string>` to every header that names `std::string` directly. Apply the same self-containment check to all demonstration headers.

7. Quantitative units are partly illustrative but not consistently defined

Classification.

Implementation ambiguity; minor.

Location.

Decorator listing, lines 237–261; Strategy output listings, lines 765–784.

Problem.

The Decorator prices 2.00 and 0.40 have no declared currency or unit. The Strategy distances and times do have units, but they are fixed literals rather than computed values. The light level is correctly described as a percentage in both prose and output.

Why it matters.

Undefined monetary units impede interpretation, while literal route estimates can appear empirical despite having no calculation or data source.

Suggested fix.

Define the beverage price unit and, if this is more than a toy calculation, represent money in integer minor units or a decimal type. Mark route numbers as sample output or remove them until they are derived from inputs.

Coverage, consistency, and submission findings

8. The Decorator chapter does not meet the per-pattern assignment coverage

Classification.

Definite completeness problem; major.

Location.

Seminar/Requirements.md, items 1–10 under “For each pattern”; main.tex, complete Decorator chapter, lines 178–273.

Problem.

Unlike the Facade and Strategy chapters, Decorator has no 5–8 question quiz, no separate general and problem-specific class diagrams, and no developed explanation of web and mobile applications. Its naive solutions are prose-only despite the report objective of comparing naive and pattern-based C++ designs. The workspace also contains no standalone compilable Decorator source parallel to the Facade and Strategy source directories.

Why it matters.

The assignment states that these elements are required for each pattern. The present imbalance makes the report look assembled from sections at different completion levels and can directly cost rubric credit.

Suggested fix.

Expand Decorator to the same evidence level as the other chapters: include a naive C++ example, one general diagram and one beverage diagram, a complete tested implementation, explicit pros and cons, explained web/mobile uses, and a 5–8 item quiz with an answer key.

9. Submission-status statements conflict with visible unfinished content

Classification.

Definite internal inconsistency.

Location.

“Seminar Group Members,” lines 89–118; “Submission Checklist,” lines 1019–1032; “Member Contribution Record,” lines 1042–1047.

Problem.

Responsibilities and contribution fields still contain bracketed instructions such as “[Enter contribution percentage or summary].” The appendix says remaining artifacts are maintained separately, but `Changes.md`, explicitly required by `Seminar/Requirements.md`, is absent from the workspace at review time.

Why it matters.

These are visible signs of an unfinished submission and contradict the checklist’s reassuring wording.

Suggested fix.

Replace every placeholder with final verified information. Convert the appendix from assertions to a checkable status list, and mark an item complete only after the corresponding file or form exists and has been reviewed.

10. Citation use does not match the reference list or the breadth of factual examples

Classification.

Definite citation-integrity problem, with some claims needing external verification.

Location.

Pattern-intent and application sections throughout; references, lines 1049–1074.

Problem.

Only `gof` is cited in the body, and only for Facade and Strategy. The Decorator definition is uncited. The entries `headfirst`, `refactoringguru`, `sourcecmaking`, and `cpp` are never cited. Broad examples such as Python function decorators, API Gateways, ORMs, camera APIs, and authentication/payment strategies are presented without sources or without indicating that some are analogies rather than canonical Gang of Four instances.

Why it matters.

Unused references do not establish support, and loose real-world analogies can blur the precise intent of the patterns.

Suggested fix.

Cite the source used for each formal intent, including Decorator. Cite the C++ standard where version-specific claims matter. Either support application examples with authoritative references or qualify them as illustrative analogies. Remove genuinely unused bibliography entries and add access dates to web references if required by the chosen style.

Meaningful editorial issues

Internal reviewer instructions.

The “Reviewer focus” sections in the Facade and Strategy chapters read as drafting or peer-review prompts rather than report content, and Decorator has no parallel section. Remove them from the final submitted report or move them to presenter notes. Their presence weakens the report’s audience and section-to-section coherence.

Algorithm terminology.

In the Strategy chapter, use “strategy stub,” “demonstration behavior,” or “runtime-dispatch example” until actual route computation exists. This local wording change is necessary for technical accuracy, not merely style.

Participant naming.

The first definition uses “Concrete Component” and “Concrete Decorator,” while code and conventional UML identifiers often use `ConcreteComponent` and `ConcreteDecorator`. Either form is fine, but distinguish prose role names from class identifiers consistently.

Proofing sweep

The bundled mechanical scan was run against the compiled manuscript. Its double-period hits were table-of-contents leader dots rather than punctuation errors; spot-checking found no genuine duplicated comma, semicolon-capitalization, malformed “into ... into” frame, product-name capitalization, or `arctan(x/y)` branch issue. The reference list also contains no obvious duplicated punctuation or malformed quotation punctuation. The following genuine, high-confidence defects remain:

- **Location:** Seminar/Strategy/src/strategy/BicyclingStrategy.h, line 10. **Problem:** “elevation elevation profile optimization” repeats a word. **Fix:** change it to “elevation-profile optimization” (or “elevation profile optimization”).
- **Location:** member tables, lines 99–113. **Problem:** bracketed example and entry instructions remain in final-facing prose. **Fix:** replace them with verified responsibilities and contributions; do not merely delete the fields.
- **Location:** Facade home-theater diagram. **Problem:** the relation label “wraps” imports Decorator terminology and conflicts with the adjacent explanation. **Fix:** relabel every such edge “uses,” “coordinates,” or “delegates to.”

Highest-priority fixes

1. Make the Strategy claims match the implementation: either implement input-dependent route results or explicitly present the classes as stubs.
2. Correct all diagrams to preserve ownership, reference, pointer, and constness semantics, and remove the UML composition/aggregation ambiguity.
3. Complete the Decorator chapter and provide tested source so every explicit per-pattern assignment requirement is met.
4. Correct the const explanation, state or align the C++ language version, and make the headers self-contained.
5. Replace submission placeholders and reconcile the checklist with actual artifacts.
6. Add claim-level citations, remove unused references, and qualify examples that are only pattern analogies.

Items requiring external verification

The local manuscript and source files are sufficient to establish the internal errors above. Before submission, externally verify the taxonomy of the claimed real-world examples—especially Python function decorators, API Gateways, ORMs, camera APIs, and middleware—against authoritative documentation or design-pattern literature. These may be useful analogies, but the manuscript currently does not establish that each is a canonical implementation of the named Gang of Four pattern. Also verify the required citation format and whether access dates or URLs are mandatory for the course submission.